Chains in LangChain

1. Introduction

When building AI applications, one single LLM call is often **not enough**.

Real applications need:

- Multiple steps
- Data flow between steps
- Organized processing

👉 **Chains** solve this problem.

---

2. Recap

You already know:

- LLMs take input → give output
- Output parsers structure responses

But real-world tasks are like:

👉 "Take input → process → refine → store → respond"

❌ Doing this manually is messy

✅ Chains help automate this flow

---

3. What & Why (Core Concept)

What is a Chain?

A **Chain** is:

A sequence of steps where the **output of one step becomes the input of the next**

💡 **Simple Analogy:**

Think of a **factory assembly line**:

- Step 1 → prepares raw material
- Step 2 → processes it
- Step 3 → final product

👉 That entire pipeline = **Chain**

Example:

User input:

"Write a blog about AI"

Chain flow:

- Generate outline
- Expand each section
- Format into blog

👉 Final output = complete blog

Why Chains are Important:

1. Break complex tasks into steps
2. Improve accuracy
3. Reuse components
4. Build real-world applications
5. Better control over LLM behavior

---

4. Simple Chain

✅ **What it is:**

A **single-step chain** (one input → one output)

Example:

prompt → LLM → output

💡 **Example Task:**

Input:

"Explain AI"

Output:

"AI is..."

✅ **Use Cases:**

- Chatbots
- Simple Q&A
- Text generation

❌ **Limitation:**

- Cannot handle multi-step problems

---

5. Sequential Chain

✅ **What it is:**

A chain where steps happen **one after another**

👉 Output of step 1 → input of step 2 → input of step 3

Flow:

Input → Step1 → Step2 → Step3 → Final Output

Example:

Task: Write a blog

1. Generate title
2. Create outline
3. Expand into full article

💡 **Real Example:**

Input:

"AI in healthcare"

Step 1 → Title:

"The Future of AI in Healthcare"

Step 2 → Outline:

- Introduction
- Applications
- Challenges

Step 3 → Full blog

✅ **Benefits:**

- Structured workflow
- Easy to debug
- Better quality output

⚠️ **Important:**

- Each step depends on the previous one
- If one step fails → chain breaks

---

6. Parallel Chain (Correcting timestamp typo)

✅ **What it is:**

Multiple tasks run **at the same time (independently)**

Flow:

→ Task1 →

Input → Task2 → → Combine Output

→ Task3 →

Example:

Input:

"AI"

Parallel tasks:

- Generate summary
- Generate keywords
- Generate questions

💡 **Output:**

```
{
 "summary": "...",
 "keywords": ["AI", "ML"],
 "questions": ["What is AI?"]
}
```

✅ **Benefits:**

- Faster execution
- Independent processing
- Efficient for multiple outputs

⚠️ **Important:**

- Tasks do NOT depend on each other

---

7. Conditional Chains

✅ **What it is:**

Chain that runs **different steps based on conditions**

💡 **Think like:**

👉 "If this → do A

👉 Else → do B"

Example:

Input:

"Translate this to French"

Condition:

- If translation → use translator chain
- If summarization → use summary chain

Flow:

Input → Check Condition → Choose Path → Output

Real Example:

User input:

"Summarize this text"

System:

- Detect intent = summarization
- Run summarization chain

✅ **Benefits:**

- Dynamic behavior
- Smarter applications
- Flexible workflows

🔥 **Putting It All Together (Big Picture)**

🧠 **Types of Chains:**

1. **Simple Chain**
    - One step only
2. **Sequential Chain**
    - Step-by-step processing
3. **Parallel Chain**
    - Multiple independent tasks
4. **Conditional Chain**
    - Decision-based flow

🔄 **Real-World Example (Full System)**

Task: AI Content Generator

Input:

"AI in education"

Chain Design:

1. **Sequential Chain**
    - Generate title → outline → article
2. **Parallel Chain**
    - Extract keywords
    - Generate summary
3. **Conditional Chain**
    - If user asks for short → summarize
    - Else → full article

Final Output:

- Blog article
- Keywords
- Summary

⚠️ **Important Points to Remember**

1. Chains = **workflow of LLM operations**
2. Help break complex problems into steps
3. Improve output quality
4. Enable real-world applications
5. Can combine multiple chain types

✅ **Final Summary**

- **Chain = Sequence of steps using LLMs**
- Types:
    - Simple → single step
    - Sequential → step-by-step
    - Parallel → multiple tasks at once
    - Conditional → decision-based
- Chains are essential for:
    - Automation
    - AI apps
    - Complex workflows
